LMU ☀ CMSI 3802

LANGUAGES AND AUTOMATA II

Syllabus • Spring 2025

4 semester hours
Mondays and Wednesdays 11:50 a.m. – 1:30 p.m. in Pereira 140
Instructor: Ray Toal, Doolan 110, rtoal@lmu.edu, +1 310.338.2773
Teaching Assistant(s): Natalie Lau
Slack channel (lmucs.slack.com): cmsi3802-spring-2025

Learning Outcomes

Students will:

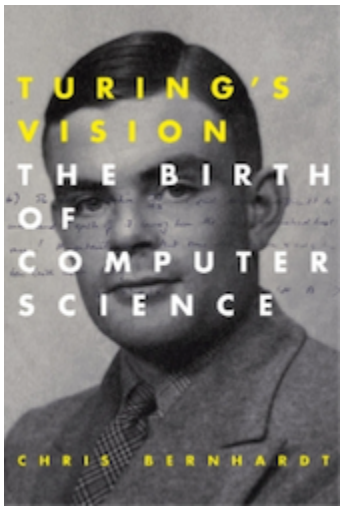
- Gain a working knowledge of two major theories of computation—language theory and automata theory—and a brief familiarity with two others—computability theory and complexity theory;
- Become accomplished in language processing techniques, compiler theory, compiler design techniques, and the notion of intermediate languages and virtual machines;
- Increase their software development expertise by building a complex system (a compiler) using a modern tool set including Node, Ohm, and c8, as an open source project hosted on a public repository; and
- Be able to tell the story of the birth and subsequent evolution of Computer Science.

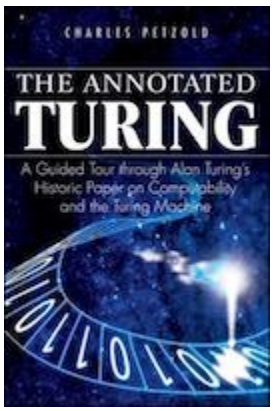
Prerequisites

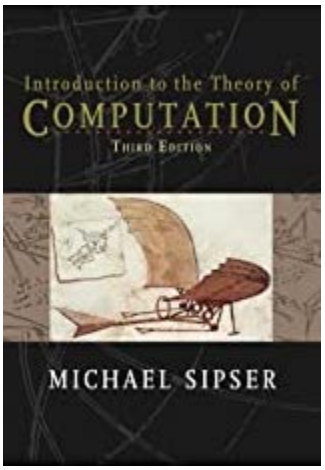
Fluency in a high-level programming language such as JavaScript, Python, or Java. Courses (or demonstrated competency) in Programming Languages and Systems Organization.

Readings

As this course deals with fundamental theories of computer science, you will be expected to keep up with essential readings. Do not skip them!









You are expected to read the entirety [Bernhardt], supplemented with readings from [Sipser]. There will be assigned chapters from [Mogensen]. You must also read/watch the items in red from [Robins].

Details on the required readings and a handful of suggested additions follow:

- Books on Theory and History
    - Chris Bernhardt, Turing's Vision: The Birth of Computer Science, MIT Press, 2016.

    - Charles Petzold, The Annotated Turing, Wiley, 2008.Michael Sipser, Introduction to the Theory of Computation, 3rd Edition, Cengage, 2012. (pdf)
- Programming Language Texts that Cover Implementation
    - Robert W Sebesta, Concepts of Programming Languages, 11th Edition, Addison-Wesley, 2015.
    - Michael L. Scott, Programming Language Pragmatics, Morgan Kauffman, 2009.
- Books on Compilers
    - Torben Ægidius Mogensen Basics of Compiler Design
    - Dick Grune and Kees van Reeuwijk, Modern Compiler Design, Springer, 2012.
    - Keith Cooper and Linda Torczon, Engineering a Compiler, Morgan Kauffman, 2011.
    - Aarne Ranta Implementing Programming Languages: An Introduction to Compilers and Interpreters, College Publications, 2012.
    - David Galles, Modern Compiler Design, Scott/Jones, 2004.
    - Niklaus Wirth, Compiler Construction.
    - Steven Muchnick, Advanced Compiler Design and Implementation.
- Books on Designing Your Own Programming Language
    - Marc-André Cournoyer, How To Create Your Own Freaking Awesome Programming Language.
    - Terrance Parr, Language Implementation Patterns: Create Your Own Domain-Specific and General Programming Languages, Pragmatic Programmers, 2010.
- Papers on Compiler Writing
    - Jack Crenshaw, Let's Build a Compiler.
    - Dipanwita Sarkar, Oscar Waddell, and R. Kent Dybvig, A Nanopass Framework for Compiler Education.
- James Hague's Commentaries
    - Want to Write a Compiler? Just Read These Two Papers
    - A Forgotten Principle of Compiler Design

    - On the Madness of Optimizing Compilers
- JavaScript and Node
    - Any or all of Axel Rauschmayer's JavaScript books (all free online)
    - The Learning Section on the Node.js site (full of good stuff)
- Ohm
    - Ohm home
    - The Modular Semantic Actions paper
- Web Resources on Compilers and Parsing
    - Papers and articles referenced from Bryan Ford's Packrat Parsing and PEG page
- x86 Architecture Resources
    - Intel 64 and IA-32 Software Developer's Manuals
    - Sandpile
    - Agner Fog's Software Optimization Resources
- ARM and M1 Resources
    - Wikipedia article on ARM
    - Wikipedia article on the Apple M1
- Gabriel Robins's Great Readings for Researchers and Scholars. Read all of the items in red.

Various papers and readings will be assigned throughout the course (including my own course notes, practice problems, and sample code). These readings will be posted on individual assignment pages. If you have projects or papers to work on, you'll have to find some additional readings on your own. Make sure you take the time for effective self-study. Take advantage of classmates and friends; the computing industry is one of the most collaborative fields in which to work, and your course experience should reflect this.

Workload

In accordance with the LMU Credit Hour Policy, this 4-unit course will require 12 hours of work per week (including the time spent in lecture and lab).

Assignments and Grading

You'll have several homework sets containing in-depth theoretical questions and non-trivial programming problems, and quizzes and a final exam with less difficult material. In this class you will write a complete compiler. You will have, as a starting point, an existing compiler provided by the instructor for a simple programming language. You may either extend the compiler or start from scratch. To keep you on track, the project will be turned in in pieces, specifically as part of the homework assignments for the semester. The assignments will also contain some theoretical problems for you to work out, too. You have to write a report documenting the architecture and design of the compiler; however, your writeup may appear on your wiki as opposed to a stodgy PDF. To help prepare you to meet industry expectations for college graduates, programming assignments will sometimes be required to be placed in version-controlled public repositories (most students tend to prefer GitHub). Exams may cover material from lectures *not previously assigned* for homework, so take care to learn concepts and not just memorize technical steps and recipes to answer questions.

Occasionally, you may be permitted to work in groups of students; however, while only one solution set is turned in per group, both students are responsible for understanding *all* of its content and may be asked at any time for an oral explanation of any solution. Collaboration with other groups is fine but must be limited: you may share ideas and approaches but not solutions. You **must** acknowledge any help received. Academic dishonesty may result in expulsion; be certain your work meets the standards set forth in the LMU Honor Code.

Your final grade will be weighted as follows:

Homework sets, including project checkpoints
50 pts

Quiz 1
10 pts

Quiz 2
15 pts

Final Exam
25 pts

Letter grades are figured according to the usual scale: 90% or more of the total points guarantees you an **A**, 80% a **B**, 70% a **C**, and so on. Note the word *guarantee*: 82 points will earn you *at least* a **B-**; you might still get an **A** if 82 is or is near the top score. The lower bounds ensure grades measure your achievement of the learning outcomes, and can never punish you because you did very well but on the low end of a class full of high-achievers.



To ensure a degree of fairness for those who consistently perform timely course work, and to encourage everyone to pace themselves properly in completing assignments, late work is normally penalized 15% per 24-hour period. If there is an issue that prevents you from submitting an assignment on time (e.g., excellent surf conditions, personal or family issues, sickness, conference attendance, job interviews, a family ski trip, or personal emergencies), let the instructor know ahead of time.

Where assignments involve programming, the *quality* of your code, not just its *correctness*, will play a large part in determining your grade. Please refer to

these resources and notes on clean code for information on expectations of code quality. Appearance of the grading policy in this syllabus constitutes fair warning of the consequences of poorly written code.

If the course has a contribution score, it will be computed by awarding you 1 point every time you: (1) correct me during class, (2) ask a profound question during class (where profound elevates the discussion in some sense), (3) answer a question during class as the first response, (4) post a good question on the class Slack channel, (5) post a detailed answer to another student's question on the Slack channel, (6) recommend an article or video or tutorial (on the Slack channel) directly related to the course material. There is an expectation of 10 points throughout the semester, well-spaced. (Please do not dump 10 video recommendations on Slack during the first week of class and then shut down.)

Important Dates

When	What
2025-01-20 (Monday)	No class (Martin Luther King, Jr. Day)
2025-01-31 (Friday)	Homework 1 Due
2025-02-21 (Friday)	Homework 2 Due

2025-02-24 (Monday)	Quiz 1
2025-02-28 (Friday)	Midterm deficiency grades due
2025-03-03 (Monday)	No class (Spring Break Monday)
2025-03-05 (Wednesday)	No class (Spring Break Wednesday)
2025-03-30 (Sunday)	Homework 3 Due
2025-03-31 (Monday)	No class (Cesar Chavez Day)
2025-04-09 (Wednesday)	Quiz 2
2025-04-11 (Friday)	Last day to drop
2025-04-16 (Wednesday)	No class (Easter Break Wednesday)

2025-04-22 (Tuesday)	Homework 4 Due
2025-05-06 (Tuesday)	Homework 5 Due
2025-05-12 (Monday)	Final Exam

Student Rights and Responsibilities

You have the right to:

- A syllabus with stated learning outcomes
- Have those learning outcomes addressed
- Chats with the instructor during office hours
- Have questions answered via email (one-day turnaround max)
- Video and text chats with the instructor
- Feedback on your work
- A challenging experience (you should be working a little outside your comfort zone)
- All lecture sessions recorded and stored online for you to access
- An experience beyond what you would get from books, websites, online tutorials, and discussion forums
- Feel like you belong (if you are experiencing impostor syndrome or stereotype threat, please see the instructor)
- Leave the course better skilled than when you came in

In return, you are expected to:

- Use the lab resources, TA time, and instructor time when you need them
- Stay on top of all course announcements, both on BrightSpace and Slack
- Follow all homework submission instructions exactly
- Watch the course recording if you miss class
- Perform work conscientiously, neatly, and in a timely manner
- Be considerate and respectful to your fellow students
- Be honest with yourself and others
- Finish the course or withdraw by the withdrawal deadline

For online courses, I recommend your camera be turned on unless you have bandwidth problems or need to step away.

Two Tips for Success

You've probably seen hundreds of these, but there are two that stand out.

> **Be aware of how students trick themselves into thinking they're learning.** Cramming, rereading, and highlighting do **not** move information from your working memory into long-term memory. Short-term working memory is not "learned." Recall is the most effective way to load your long-term memory. Please work through Nicky Case's interactive essay How to remember everything forever-ish. (Also, this article backs up the importance of recall.)
> **Use checklists**, as these are incredibly effective. Read Atul Gawande's The Checklist and think about how you apply these to your studies.

Generative AI and Your Learning

Academic honesty includes the appropriate use of technology as an aid for learning and productivity. This includes but is not limited to LLM-based generative artificial intelligence tools such as ChatGPT, Gemini, and Copilot.

Modern GenAI provides fast solutions to a variety of computing problems but must be used responsibly to get the most out of your education. GenAI will not always be helpful, especially when asked to write code for scenarios it cannot recognize. It cannot independently validate the code that it produces, and will frequently produce bugs and security vulnerabilities, making things worse for you. It will not be available for job interviews. As a student and future software professional, you have a moral and ethical responsibility to deeply understand every line of the software you author and should therefore employ GenAI in your coursework **sparingly**, especially when its use might rob you of the stretch-zone learning that comes from crafting programs on your own.

That said, there are several acceptable use cases of GenAI in programmatic assignments, including:

- Generating short documentation strings and type hints.
- Finding examples and explanations for syntactic forms (e.g., how to use the spread operator; how to assign variables directly from an array or object; how to construct a dictionary comprehension; how to unwrap an optional, and so on). Prompt the AI for the how-to's, *not* your final code.
- Interpreting error messages, as these might be pretty cryptic! Sometimes the AI can be a useful tutor when explaining things it is familiar with.
- Generating unit tests for edge cases. When it finds something you overlooked, make a mental or written note so you can find these cases on your own next time.
- Brainstorming new ideas for apps.

Remember the four As

*DO use technologies such as GenAI to **augment** ✅, **amplify** ✅, and **accelerate** ✅ your learning; NEVER use technology to **avoid** ❌ learning.*

Be responsible!

Generally, using GenAI and related tools without knowing what you are doing often leads to poor homework submissions that rarely achieve a grade higher than an F on their own merits, simply due to hallucinations and an inability to carry out tasks as they were assigned (even with good prompts).

Also, instructors and TAs are GenAI users themselves and will be able to easily detect inappropriate student use of these tools. The teaching staff will try to steer you back on track should we feel like you are giving up on your learning.

If you would like to use GenAI appropriately as a learner, see this short introductory article. If you are a TA, see this article by Jeff Olson on how to help teach students to use GenAI.

Topics

What do we need to know to become a computer scientist?

- UNIT 0—COMPUTATION, INCLUDING A BRIEF HISTORY: Definitions of information, computation and language. Why the study of computing is the study of languages and automata. Computing as pop culture; Matching and reckoning; The usefulmess of numerals; Early computing recipes and machines; Formalisms in mathematics; Church, Gödel, and Turing; Modern machines and programming languages; Compilers, optimizations, human factors, and modern trends. Computing as a human experience. Big ideas. A sneak peak at the four theories.
- UNIT 1—LANGUAGE DESIGN AND SPECIFICATION: The language design process; Review of several programming languages, concepts, paradigms, and features. The surprising central importance of abstract syntax. How to sketch and prototype in your design, making use of design tools such as Esprima and Ohm. Experimentation with different syntactic

forms; A look at sample languages (Astro, Bella, Carlos, Dax, Ekko), some with formal syntax and semantics.

- UNIT 2—WRITING SIMPLE INTERPRETERS AND COMPILERS: The architecture of compilers and interpreters. How to organize a project; Writing an interpreter for Astro; Writing a compiler for a more complex language, Bella; Writing a compiler for an even more complex language, Carlos. Explanations and concrete examples of each phase of a compiler. Syntax processing; Static analysis (Names, scopes and bindings; Object lifetimes); Optimizations (Data Flow Analysis; Peephole Optimizations; Constant Folding, Common Subexpression Elimination, Copy Propagation, Strength Reduction; Loop optimizations; Induction Variable Elimination; Inline and closed procedures.); Basic translation strategies.
- UNIT 3—THE FOUR MAJOR THEORIES OF COMPUTATION: What are theories and why should we study them? **Language Theory**: Symbols, alphabets, and languages; Grammars; Classes of languages; The main concerns of programming language theory (PLT); Syntax of real programming languages: ambiguity, operator precedence, operator associativity; Abstract syntax. **Automata Theory**: The Turing Machine and many variations; Other models of computation, including Finite Automata, Pushdown Automata, Post Canonical Systems, semi-Thue Systems, and especially Stack and Register Machines. **Computability Theory**: Limits of computation; Church-Turing Thesis; A more detailed language classification; Undecidable problems; RE and co-RE; Rice's Theorem. **Complexity Theory**: P=NP and the Complexity Zoo.
- UNIT 4—APPLICATIONS: Parsing Theory. Intermediate languages and virtual machines (e.g., JVM, CLR, LLVM, WebAssembly); Machine and assembly languages; x86, MIPS, ARM; Assembly language programming techniques, such as dealing with limited registers. Back-end compiler architecture; Code generation issues (Basic blocks, Traces, Liveness Analysis, Register Allocation); Construction of executable code and libraries. Run-time system organization (the stack, the heap, garbage collection); Translation techniques for control flow (expressions, assignment, sequencing, selection, iteration, recursion), declarations,

basic types (arrays, records, strings, pointers) and type checking, subroutine calls and parameter passing; implementation of higher-order functions and closures; Type checking; Heap management. Machine-Dependent Optimization: Pipelining and Scheduling.